

WISEConv: A Worklist-Driven Masked-Convolution Runtime for GPUs

Anonymous Author(s)

Abstract

Masked convolution promises to cut the per-frame inference latency of vision models by computing only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio and fail to convert sparsity into proportional latency reduction. We present WISEConv, a masked-convolution runtime that derives an active-output worklist from the upstream mask and uses it to drive the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by restricting computation to active positions while retaining the regular dense datapath. To amortize construction overhead and keep the compute pipeline fully utilized despite per-frame-varying worklist lengths, WISEConv co-designs construction and consumption: compatible operators reuse the constructed worklist without reconstruction, the worklist emission order preserves tile-local spatial reuse, and a data-driven adaptive decomposition adjusts parallel work to the activity level without host synchronization. Across four perception models and six GPU platforms (four discrete, two embedded), WISEConv reduces full-model latency by 22.1–46.5% relative to the fastest competing path and per-frame energy by 28.0–59.6% on the embedded platforms.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates the per-frame inference latency of many latency-critical vision tasks [2, 9, 37], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks active positions, and the operator computes only those. The mask may derive from event-camera increments [7, 22], temporal differences [16, 27], or learned spatial gates [20, 34]. Regardless of source, it poses one operator-level problem: compute the convolution only at the active positions of the dense feature grid.

Dense GPU convolution obtains high throughput from tiled, coordinate-driven execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position’s receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath. Neighboring coordinates share overlapping receptive fields, yielding predictable memory access and data reuse. However, unlike static sparsity in matrix multiplication, masked convolution selects positions at runtime per frame, requiring the operator to access scattered,

irregularly-overlapping neighborhoods and handle a varying workload, conflicting with tiled execution.

Existing masked-convolution systems preserve this regularity or eliminate wasted work, but not both. *Tile skipping* [7, 16, 27, 29] retains the dense pipeline and suppresses a tile only when all its outputs are inactive, so a single active position triggers computation for the entire tile. *Gather-scatter* [20, 34, 38] (as an execution path) instead extracts active positions exactly, materializes receptive fields, and scatters the computation results to the dense grid, sacrificing regularity and incurring extraction, irregular access, and writeback overhead.

We characterize this trade-off along two axes (§2.2): *throughput* (operations per second) and *useful-compute ratio* (the fraction of issued operations that produce needed outputs). In our sparsest workload (the DynConv-gated model, §5.2), the masks render 84.4% of the convolution work unnecessary, yet neither approach turns this into proportional latency reduction. Tile skipping [27] suffers a useful-compute ratio of only 49.1% and cuts latency by at most 40.6%. Gather-scatter drives the useful-compute ratio near one but sustains only 9.3% of tile skipping’s throughput, running 3.1× slower than the dense path.

Our key insight is that position-level selectivity does not require abandoning the dense convolution pipeline. A dense tile already derives its reads and writes from output coordinates, so replacing its contiguous coordinate block with an active-output worklist changes which outputs are computed without changing the per-position reduction, the dense tensor layout, or the weight layout. With this substitution, the operator runs as a dense tiled pipeline, raising the useful-compute ratio toward one without materializing input patches.

Translating this into latency reduction requires handling two challenges. First, *construction efficiency*: construction scans the mask, so its cost scales with the layer’s spatial grid, not with the active count or channel widths, while the compute it enables shrinks with both. Construction’s share of latency therefore grows as activity falls and channels thin (while enough positions remain active to keep the pipeline full), and accumulates across operators when every layer rebuilds. Second, *end-to-end worklist efficiency*: the worklist’s order and length both affect its consumption throughput. An unordered worklist scatters the input neighborhoods that adjacent work touches, lowering locality; its length falls below the dense grid and shifts with layer and input, so no fixed tiling keeps the GPU full, and because the length is device-resident, any host-side decision that depends on it

costs a synchronization on the per-frame path. Both effects sharpen as activity falls.

We present WISEConv (**W**orklist-**drI**ven **maSkEd C**onvolution), a masked-convolution runtime that fuses position-level selectivity into the dense tiled pipeline, securing both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. Construction is cheap and infrequent: a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse valid metadata instead of reconstructing it. Consumption is dense-like: construction emits contiguous per-tile segments ordered by the dense tile traversal, giving the compute stage tile-local structure without global sorting. Compute then decomposes parallel work adaptively according to device, layer shape, and activity, choosing each operator's configuration from offline calibration. For temporally correlated inputs, it tracks activity across frames and sustains GPU utilization without host synchronization.

This paper makes the following contributions:

- We characterize masked-convolution latency along two axes, useful-compute ratio and throughput, exposing why existing work does not translate upstream sparsity into proportional latency reduction.
- We design WISEConv, which fuses position-level selectivity into the dense tiled convolution pipeline and co-designs worklist construction and consumption to secure both axes jointly.
- We evaluate WISEConv on four perception models across four discrete GPUs and two Jetson platforms. WISEConv reduces full-model latency relative to the fastest baseline by 34.2–46.5% on the discrete GPUs and 22.1–38.6% on the Jetson platforms, and cuts per-frame energy over dense convolution by 28.0–59.6% on the Jetson platforms.

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [2, 31, 35, 37, 41], where per-frame inference latency bounds system responsiveness [8–10, 17]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [7, 22], temporal residuals between video frames [3, 16, 26, 27, 36], or learned input-dependent gates [12, 20, 34, 38]. The event and temporal policies mark positions that changed since the previous frame; the learned gates mark positions the task treats as relevant. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution keeps the tensors and arithmetic of dense convolution and restricts only which output coordinates it computes. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{w} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k , and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \dots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0, 1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0, 1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

M_{out} thus fixes which positions of $\mathbf{y}$ the operator computes, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{w}$. The active ratio $\alpha = \|M_{out}\|_0 / |\mathcal{G}|$, the active fraction of the grid, is the intrinsic sparsity the policy exposes.

An execution path issues computation over a sequence of position slots C , where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non- $\perp$ slot $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{w}_i \mathbf{x}[\mathcal{N}_i(p)], \quad (2)$$

where $\mathcal{N}_i(p)$ is the i -th position in $\mathcal{N}(p)$ and $\mathbf{w}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. Positions the path does not compute retain their existing value in $\mathbf{y}$: masked convolution updates the output in place, so $\mathbf{y}[p]$ for $p \notin \{c \in C : c \neq \perp\}$ carries over from the previous frame (event and temporal policies) or from a one-time dense initialization (spatial gating). Coverage is therefore the correctness condition: every position the policy marks as changed is recomputed, and every other position is already correct by construction. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and the required work, while C is determined by the path.

2.2 The Sparsity-to-Latency Gap

Existing systems execute a mask along one of two paths. *Tile skipping* [7, 16, 21, 26, 29, 36] inherits the dense pipeline's tile-based computation, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [27] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [20, 38] instead extracts exactly the active coordinates, computes, and scatters them back,

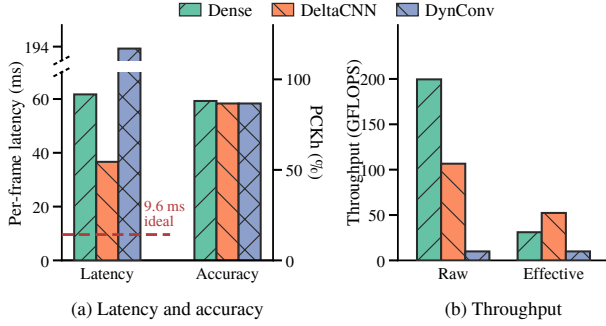


Figure 1. Statistics of DynConv [34], a pose estimator with learned spatial gating, on the MPII human-pose dataset [1] and Jetson AGX Orin, comparing the native cuDNN path (Dense) against the two sparse paths. (a) End-to-end average per-frame latency (left axis; the dashed line marks the dense latency scaled by the active ratio aggregated across layers) and PCKh accuracy (right axis; PCKh evaluates the percentage of keypoints localized within half the head size). The sparse paths apply the same mask, so their accuracy coincides. (b) Raw throughput T and effective throughput T_{eff} , aggregated across the model’s convolution layers. The dense useful-compute ratio equals the active ratio set by DynConv’s spatial gating ($\alpha \approx 15.6\%$).

removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [34] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

We measure each path by two quantities. Its useful-compute ratio is the fraction of issued position slots (including $\perp$) that carry an active output,

$$\eta = \frac{\|M_{\text{out}}\|_0}{|C|} \leq 1. \quad (3)$$

Its throughput is the rate at which it processes issued FLOPs,

$$T = \frac{2|C|k^2C_{\text{in}}C_{\text{out}}}{t}, \quad (4)$$

where t is the per-frame latency, including all auxiliary work the path performs to enable computation (mask processing, coordinate generation, auxiliary passes). Because $|C|$ counts only the convolution’s issued position slots, auxiliary costs reduce T without changing η . The leading factor of 2 counts each multiply-accumulate as two FLOPs. For the required $2\|M_{\text{out}}\|_0k^2C_{\text{in}}C_{\text{out}}$ FLOPs, latency is set by the **effective throughput** $T_{\text{eff}} = \eta T$, the rate at which a path computes needed outputs.

Figure 1 traces this on Jetson AGX Orin for the sparsest workload in our evaluation. The policy marks 84.4% of the convolution work unnecessary, and the sparse paths discard it with negligible accuracy loss, so the remaining work

sets a 9.6 ms latency floor (Fig. 1a, dashed). Neither path approaches it: tile skipping (DeltaCNN [27]) reaches 36.6 ms, still 3.8 $\times$ the floor, and gather-scatter, even with DynConv built to amortize its overhead, runs at 193.5 ms, far slower than dense convolution itself. The gap traces to low effective throughput on both paths. Tile skipping holds throughput near half of dense but at a useful-compute ratio of only 49.1%; gather-scatter lifts the useful-compute ratio near one, yet extraction and data movement collapse its throughput to 5.0% of dense (Fig. 1b). Neither path efficiently converts the GPU’s capability into effective throughput for masked convolution.

2.3 Distinction from Related Sparse Execution

Related sparse-execution work differs from WISEConv’s masked convolution in representation, operator, sparsity type, or platform. 3D sparse convolution (SpConv v2 [30], TorchSparse++ [32, 33], Minuet [39], MinkowskiEngine [6], submanifold sparse convolution [15]) is built to organize computation over an unordered list of active voxels, constructing coordinate maps that route irregular input coordinates to outputs; its input is sparse by construction rather than a dense grid under a runtime mask, so it is related infrastructure rather than a baseline. Sparse matrix multiplication (DTC-SpMM [11], MaxK-GNN [28], Sputnik [13]) reorganizes a sparse matrix operand so the dense datapath can consume it; its sparsity lives in the matrix structure of a different operator, not in a spatial mask over a convolution grid. Structured weight sparsity (2:4 tensor cores [24]) and sparse-format compilers (TACO [19], SparseTIR [40]) exploit a static structural sparsity fixed at compile time in the weights or a known format, rather than a mask discovered at runtime over the spatial grid. DPACS [14] resolves similar dynamic spatial sparsity in CNN feature maps, but on a custom accelerator instead of a commodity GPU.

3 WISEConv Design

WISEConv co-designs worklist construction and consumption so that selective execution retains a dense-compatible inner datapath while scheduling every active output. This section states the design thesis and overviews the two-stage pipeline (§3.1), then describes construction (§3.2) and computation (§3.3).

3.1 Overview

As characterized in §2.2, masked-convolution latency is governed by effective throughput ηT , the product of throughput T and the useful-compute ratio η . No existing approach jointly optimizes both axes: tile skipping retains the dense tiled pipeline but sacrifices η ; gather-scatter drives η near one but collapses T in the batch-1 latency-bound regime. In both cases, the coupling runs through the *position source*: tile skipping can raise η only by shrinking tiles, directly degrading datapath efficiency; gather-scatter achieves tight

schedules only by materializing extracted data, collapsing T . The design thesis of WISEConv is that replacing the position source with a worklist cuts this position-source coupling: the schedule's tightness no longer taxes the reduction datapath, and the reduction datapath no longer constrains the schedule.

WISEConv replaces the contiguous position source of a dense tiled convolution with a worklist $\mathcal{W}$ of scheduled output coordinates that covers at least every active output position. The worklist's length $n_{\mathcal{W}}$ is determined on device by construction. The operator runs in two stages: a *construction stage* that converts the mask into the worklist, and a *compute stage* that drives a tiled convolution from it. The compute stage issues positions in tiles of B_M entries, so the total issued count rounds up: $|C| = B_M \lceil n_{\mathcal{W}} / B_M \rceil$ position slots, where trailing slots are $\perp$ (padding that carries no output coordinate). Under this pipeline, η decomposes into two factors:

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{cov} \text{ (schedule tightness)}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{B_M \lceil n_{\mathcal{W}} / B_M \rceil}}_{\eta_{pack} \text{ (position-axis packing)}} \quad (5)$$

η_{cov} measures how tightly construction's schedule covers the exact mask; η_{pack} measures how efficiently compute packs worklist entries into its B_M -wide tiles.

Construction determines the scheduled coordinate set and its local organization. It must produce a worklist cheaply and avoid redundant rebuilds across compatible operators, which answers the construction efficiency challenge. It must also emit the worklist with enough spatial structure for compute to consume it with locality (§3.2). Compute determines how that schedule is reduced and parallelized: it must consume the worklist without a host readback of its length, and adapt its parallel decomposition to varying scheduled density (§3.3). Together with construction's spatial structure, these answer the end-to-end worklist efficiency challenge. Construction and reuse determine η_{cov} ; the tile configuration chosen by adaptive decomposition determines η_{pack} . Neither stage alone controls η .

Four co-designed components realize this pipeline: *tile-local construction* produces the schedule, *cross-operator reuse* amortizes its cost, *persistent worklist-driven compute* consumes it without host synchronization, and *adaptive decomposition* selects the tile configuration that maximizes effective throughput per density level. These components share one operator-level invariant: no stage on the per-frame path blocks on the host. The worklist and its length stay on the device, and the one host-bound quantity, the input activity count, transfers asynchronously (§3.3.2).

Two of these components trade one factor of η against T ; the other two do not. Reuse trades $\eta_{cov} < 1$ for fewer construction passes, removing a channel-independent fixed cost from t ; adaptive decomposition trades η_{pack} against per-slot

efficiency to maximize $\eta_{pack}T$. Construction sets η_{cov} to one at every rebuild point rather than trading it, and persistent compute changes only how the scheduled slots are reduced, so neither gives up η to gain T . A weaker residual coupling remains: position-axis quantization ties both η_{pack} and per-slot efficiency to B_M ; adaptive decomposition navigates this trade-off at runtime (§3.3.2).

3.2 Construction

Construction converts the upstream mask into a worklist before compute begins. The following subsections describe the two key mechanisms: local compaction that produces contiguous per-tile segments without a device-wide ordered-compaction pass (§3.2.1), and a cross-operator reuse protocol that amortizes construction across compatible operator sequences (§3.2.2).

3.2.1 Tile-Local Worklist Compaction. Any worklist construction must scan the output grid to identify active coordinates; the design question is how to organize the result. A globally stable compaction can retain full-grid traversal order, but producing global offsets requires device-wide coordination or an additional pass (e.g., a prefix scan that propagates segment bases). An alternative is to extract active positions along with their input neighborhoods (gather-scatter extraction), but this materializes feature data proportional to $\alpha H_{out} W_{out} \cdot k^2 C_{in}$, far exceeding a metadata-only pass.

WISEConv needs only local coordinate structure, not a total order, and therefore fuses mask propagation with independent local compaction in a single metadata pass, avoiding both the coordination of global compaction and the data movement of gather-scatter extraction. Construction partitions the output grid into fixed spatial tiles of $B_T \times B_T$ positions ($B_T = 16$) and processes each tile independently (Algorithm 1). Within a tile, every output position checks whether any input in its $k \times k$ receptive field is active (the condition of Eq. 1), which also produces the output mask for downstream propagation. Each warp counts its active positions and publishes only that count to tile-local shared memory; one atomic reservation per tile converts the tile's total into a base offset, from which each warp derives its own by the prefix of preceding warp counts, and each warp then writes its coordinates directly into the tile's worklist range in traversal order.

The launch grid follows from the layer geometry ($\lceil H_{out} / B_T \rceil \times \lceil W_{out} / B_T \rceil$ blocks), so construction needs no device-resident quantity and the host launches it without synchronization. Atomic reservation leaves the inter-segment order nondeterministic, but each active coordinate is reserved exactly once regardless of placement, so correctness does not depend on segment order.

The worklist buffer is pre-allocated at initialization to the output grid upper bound $|\mathcal{G}| = H_{out} W_{out}$ entries (one int32

[TODO] WISEConv overview. Left: dense input feature tensor and input mask. Middle: local construction emits the output mask, worklist coordinates, and device-resident worklist length. Right: worklist coordinates replace the contiguous position source of tiled convolution; channel and kernel reduction proceeds as in the dense path.

Figure 2. The WISEConv two-stage pipeline. Construction discovers scheduled output coordinates and compacts local segments into the worklist. Compute drives a tiled convolution from the worklist while retaining regular channel and kernel reduction.

coordinate each, totalling $4|\mathcal{G}|$ bytes). This avoids a device-wide two-pass scan (count then allocate) or a host-side allocation decision, both of which would require knowing the active count before construction writes. The additional memory per rebuild point is on the order of the output feature map size along one channel, a fixed overhead independent of activity.

Locality. All coordinates in a segment lie within one $B_T \times B_T$ spatial tile, so the receptive fields they address fall inside a bounded neighborhood: gather addresses are clustered within a segment. Across segments no such bound holds, because atomic reservations place segments in non-deterministic order. How many segments a B_M -wide compute tile draws from depends on segment length: with $\bar{s}$ the mean number of active positions in a non-empty construction tile (empty tiles reserve nothing and emit no segment), a compute tile spans about $B_M/\bar{s}$ segments, and in the limit of one active position per tile it spans B_M . Locality is tile-local by construction, not a global ordering property.

B_T and activity jointly set the segment length: uniformly spread activity gives $\bar{s} \approx \alpha B_T^2$, and a clustered mask raises $\bar{s}$ above this estimate by filling fewer tiles more densely. As activity falls, segments shorten and a compute tile draws from more of them. A larger B_T lengthens segments and issues fewer reservations, but the launch grid holds only $\lceil H_{out}/B_T \rceil \lceil W_{out}/B_T \rceil$ blocks, so deep layers with small output grids lose construction parallelism first. We fix B_T at build time rather than calibrate it per density level, because construction cost depends on the grid geometry alone. The response to falling activity belongs on the compute-tile axis instead: a smaller B_M spans fewer segments (§3.3.2).

Algorithm 1 Tiled Worklist Construction

Require: Input mask M_{in} , output grid $\mathcal{G}$, construction tile size B_T
Ensure: Worklist $\mathcal{W}$, output mask M_{out} , worklist length $n_{\mathcal{W}}$

```

1:  $n_{\mathcal{W}} \leftarrow 0$ 
2: for each spatial tile  $\tau$  of  $B_T \times B_T$  positions in parallel do
3:   for each position  $p \in \tau$  in parallel do
4:      $\text{active}(p) \leftarrow \exists r \in \mathcal{N}(p) : M_{in}[r] = 1$ 
5:      $M_{out}[p] \leftarrow \text{active}(p)$ 
6:   end for
7:    $n_{\tau} \leftarrow |\{p \in \tau : \text{active}(p)\}|$       {per-warp counts combined in shared memory}
8:    $\text{base} \leftarrow \text{ATOMICRESERVE}(n_{\mathcal{W}}, n_{\tau})$ 
9:   Write active  $p \in \tau$  to  $\mathcal{W}[\text{base} \dots \text{base} + n_{\tau}]$  in traversal order
10: end for
```

Dual-axis effect. Fresh construction schedules exactly the active coordinates, so $\eta_{\text{cov}} = 1$: the schedule is tight and no position slot is wasted on an inactive coordinate. The per-position reduction, tensor layout, and weight layout are unchanged from dense convolution, so along the channel and kernel dimensions the per-slot datapath is identical to the dense path. The two paths differ on the position axis: a dense tile loads a contiguous input window, while a worklist tile gathers by coordinate. Effective throughput under fresh construction is therefore $\eta_{\text{pack}} T$, with two losses relative to a dense tile of the same shape: the trailing-tile quantization ($\eta_{\text{pack}} < 1$) and the per-slot cost of gathering rather than streaming. Compute’s tile choice B_M controls both (§3.3).

Cost model. For a fixed geometry, construction scans the full output grid and performs only mask and coordinate work, yielding a cost of $O(H_{out}W_{out} \cdot k^2)$.¹ This cost is independent of the channel dimensions C_{in} and C_{out} : construction operates on mask metadata alone and never touches the feature or weight tensors. Under fresh construction, $n_W = \|M_{out}\|_0 = \alpha H_{out}W_{out}$. Compute costs $O(n_W \cdot k^2 C_{in}C_{out})$, scaling with both the active ratio and the channel dimensions. The ratio of construction to compute cost is therefore proportional to $1/(\alpha C_{in}C_{out})$: construction's share of total operator latency grows as channels thin or activity falls, and each reconstruction adds a full grid scan. This scaling holds when $n_W \gg B_M$; once the scheduled length falls to compute-tile order, compute cost is floored by the minimum tile issue ($B_M k^2 C_{in}C_{out}$), and the construction share saturates at a constant independent of α . The remaining per-layer overhead motivates the reuse mechanism below.

3.2.2 Cross-Operator Worklist Reuse. Construction cost is channel-independent and precedes compute, so consecutive operators that each rebuild the worklist pay the same spatial-scan cost regardless of the compute they enable. A $k > 1$ convolution must rebuild (the rebuild criterion below), but the operators between such convolutions need not. A convolution block interleaves its spatial convolutions with unit-kernel operators (activations, inference-time normalization, 1×1 projections, residual adds), each of which would otherwise pay its own full-grid scan even though its output grid and active set are already described by the incoming metadata. In a model with many such operators, these scans accumulate.

WISEConv tracks the exact active set and the scheduled coordinates independently. The exact mask records which outputs are active; the worklist records which coordinates the kernel will process. A compatible operator can inherit the upstream worklist without reconstruction when the incoming coordinates still name the downstream operator's output grid and cover its active set.

Compatible operators. A 1×1 convolution with unit stride, zero padding, and unit dilation maps each input position to exactly one output at the same spatial coordinate; it preserves both the grid geometry and the active set. Pointwise operators that preserve the active set (activations without thresholding, normalization at inference time) are identity on the coordinate mapping and forward the worklist unchanged. In both cases the inherited worklist equals the active set, so η_{cov} remains 1: reuse eliminates a construction pass without loosening the schedule.

A pointwise operator that narrows the active set (e.g., a thresholding activation) updates the exact output mask

¹When an upstream policy already produces the output active set directly (e.g. learned spatial gating [20, 34]), WISEConv replaces receptive-field propagation with the gate's own activity check, removing the k^2 factor from that layer's construction cost.

while retaining the incoming worklist as a superset schedule: the worklist covers all positions active under the new mask, plus positions that were active before narrowing. At operator j in a compatible sequence whose leading construction scheduled n_W coordinates, $\eta_{cov}^j = \|M_{out}^j\|_0 / n_W$ (we drop the superscript where the operator is clear); each narrowing operator reduces this ratio, and the effect accumulates along the sequence. Subsequent same-grid operators consume this worklist directly, trading $\eta_{cov} < 1$ for one fewer full-grid construction pass.

Reuse is legal only while the inherited coordinates remain a valid schedule, so WISEConv needs a rebuild criterion. A natural choice would rebuild when η_{cov} drops below a cost-based threshold, but evaluating $\|M_{out}^j\|_0$ at runtime requires reading a device-resident quantity back to the host, reintroducing the per-operator synchronization that the persistent compute design eliminates (§3.3.1). WISEConv therefore uses a statically decidable criterion: an operator reconstructs its worklist whenever the inherited coordinates can no longer serve as a legal schedule. This occurs when the consumer's output grid differs from the producer's (a change in stride or spatial dimensions), or when the operator is a convolution with $k > 1$, whose receptive field may dilate the active set beyond the inherited coordinates. The second condition is deliberately conservative: whether dilation actually adds coordinates is data-dependent, and testing it would require the same device readback the criterion avoids, so WISEConv rebuilds whenever dilation is possible. Because the criterion is geometric, a compatible sequence runs to its end even when η_{cov} has decayed, without measuring the trade at run time.

Metadata protocol. WISEConv propagates worklist metadata across operators via a fixed tuple: the feature tensor, the exact spatial mask, the worklist buffer (a coordinate array), and a device-resident worklist length giving the buffer's valid prefix. Fresh construction initializes the mask and the worklist with the same coordinate set. A reuse-compatible operator inherits the worklist directly; an operator that reduces the active set updates only the mask; an operator that selects the dense candidate executes via cuDNN but forwards the inherited worklist unchanged. Construction executes only at *rebuild points* where the inherited schedule is no longer valid; all operators within a compatible sequence inherit the worklist from the sequence's leading construction.

Dual-axis effect. On the η axis, each reused operator incurs the $\eta_{cov}^j \leq 1$ defined above, with the gap growing as narrowing operators reduce the exact mask. On the T axis, each reuse removes one $O(H_{out}W_{out})$ construction pass from t (reused operators have unit kernels, so no k^2 factor). This term is independent of the channel dimensions, so it dominates for pointwise operators, whose own compute is $O(n_W C)$. The net benefit is positive when the removed

construction cost exceeds the additional wasted compute from $\eta_{\text{cov}} < 1$; §5 measures this per workload.

Reuse in practice. In the YOLOv8 temporal path, a thresholding activation narrows the exact mask and the subsequent 1×1 convolution (same grid, unit stride) consumes the upstream worklist as a superset schedule. Construction executes once per compatible sequence rather than once per operator.

3.3 Computation

Construction delivers a worklist that covers at least every active output coordinate; the compute stage must consume it at dense-pipeline throughput. Two properties of the worklist shape the compute design. First, its length is produced on the device, so the host cannot dimension a launch grid from it without a synchronization. Second, it varies widely across frames and layers: the same output-position tiling that saturates the device at high scheduled density leaves most SMs idle at low scheduled density (underfill). The first subsection builds a compute path that consumes the worklist without host synchronization and exposes the decomposition knobs that control underfill; the second selects among those knobs per density level.

3.3.1 Persistent Worklist-Driven Convolution.

GEMM mapping. The compute kernel realizes Eq. 2 as an implicit GEMM: the M dimension spans the n_W scheduled output positions, N spans C_{out} output channels, and K spans $k^2 C_{\text{in}}$ (kernel offsets $\times$ input channels). Grouped and depthwise convolutions use the same worklist interface, each group forming an independent GEMM with proportionally smaller N and K . Work is tiled along three dimensions: B_M output positions per tile (drawn from the worklist), B_N output channels per tile, and B_K input channels per reduction step. Only the first two multiply the work count; B_K sets the reduction granularity. The total work count is $\lceil n_W/B_M \rceil \times \lceil C_{\text{out}}/B_N \rceil \times S_k$ items, where $S_k \geq 1$ is the split- K factor defined below.

Gather-at-load. For each output tile, the kernel reads output coordinates from the worklist, decomposes each into row and column indices, and computes the receptive-field base address. During the channel and kernel reduction, the kernel gathers input values directly from the dense feature tensor using these addresses (implicit GEMM). No intermediate im2col buffer is materialized; the position addresses are indirect, while channel and kernel reduction retain regular tiled traversal.

Persistent grid. In a standard dense convolution, the host knows the output size at launch time and dimensions the kernel grid accordingly. With a worklist, the output count becomes known only on the device after construction completes. A naïve approach would read the count back to the

host before each launch, but this inserts a device-to-host synchronization per operator, stalling the streaming inference pipeline. An alternative avoids the readback by launching a grid sized for the full output grid and letting excess blocks return immediately; this eliminates synchronization but still submits blocks proportional to the full grid regardless of activity.

WISEConv uses a persistent kernel design that addresses both concerns. At module initialization, the runtime queries the device for the maximum number of thread blocks that can co-reside on each SM given each kernel configuration’s resource usage (registers, shared memory). Each selected configuration launches an occupancy-sized grid whose dimensions are independent of the runtime worklist length. The grid size is bounded by device co-residency rather than by the output grid, so the submitted block count tracks device capacity instead of problem size. At kernel entry, thread 0 of each block reads the worklist length from device memory (written by the preceding construction). If the length is zero, all blocks return immediately; otherwise, each block iterates over work items in a grid-stride loop until the worklist is exhausted. An occupancy-sized grid cannot create work that does not exist: when the worklist is short, most resident blocks find nothing to iterate over and the device runs below capacity. The residual problem is a shortage of independent work, not an excess of blocks; split- K and the tile width B_M are the two levers over this shortage, and §3.3.2 sets both.

Split- K . Split- K attacks underfill along the reduction dimension: the S_k splits cover disjoint slices of K , so the arithmetic per scheduled position is unchanged and $|C|$ is independent of S_k . Each split does re-read the same B_M coordinates, and the extra blocks, atomic accumulation, and zeroing pass all enter t ; S_k therefore moves T and leaves η untouched. When $S_k > 1$, a worklist-driven zeroing pass clears y at the scheduled coordinates before the reduction begins; like the compute kernel itself, this pass reads the worklist from device memory and requires no host synchronization.

Position-axis packing. Because compute issues B_M slots per output tile, the issued count $|C|$ (§3.1) rounds up to $B_M \lceil n_W/B_M \rceil$; trailing $\perp$ slots in the last tile carry no output. This yields $\eta_{\text{pack}} = n_W / (B_M \lceil n_W/B_M \rceil)$, bounded below by $1 - B_M / (n_W + B_M)$. When $n_W \gg B_M$ the ratio approaches 1; when $n_W = 1.3 B_M$ it drops to 0.65; when $n_W < B_M$ only one tile is issued and $\eta_{\text{pack}} = n_W/B_M$, which can be arbitrarily small. At deep layers with small spatial grids and low activity, n_W enters this sub- B_M region, making η_{pack} the dominant loss in η even under fresh construction. B_M also sets per-slot datapath efficiency, so η_{pack} and T cannot be chosen independently; adaptive decomposition (§3.3.2) navigates the trade-off per density level, while η_{cov} stays fixed by construction and reuse.

Algorithm 2 Persistent Worklist-Driven Convolution

Require: Worklist $\mathcal{W}$ (device-resident), input $\mathbf{x}$, weights $\mathbf{w}$
Ensure: Output $\mathbf{y}$ (updated at worklist positions)

```

1: Precondition: if  $S_k > 1$ ,  $\mathbf{y}$  at scheduled coordinates is
   zeroed
2:  $n_{\mathcal{W}} \leftarrow$  read worklist length from device memory
3:  $N_{\text{work}} \leftarrow \lceil n_{\mathcal{W}}/B_M \rceil \times \lceil C_{\text{out}}/B_N \rceil \times S_k$ 
4: for file = block_id, block_id + grid_size, ... <  $N_{\text{work}}$ 
   do
5:    $(b_m, b_n, s) \leftarrow$  decompose tile
6:   Load  $\mathcal{W}[b_m B_M \dots \min((b_m+1)B_M, n_{\mathcal{W}})]$ , padding
     the tile to  $B_M$  slots with  $\perp$ 
7:   Decode each non- $\perp$  entry to  $(h_{\text{base}}, w_{\text{base}})$ 
8:   Initialize accumulator  $\mathbf{Acc} \leftarrow \mathbf{0}_{B_M \times B_N}$ 
9:   for each reduction tile assigned to split  $s$  do
10:     $\mathbf{A} \leftarrow$  gather from  $\mathbf{x}$  at receptive-field addresses
11:     $\mathbf{B} \leftarrow$  load corresponding weight slice
12:     $\mathbf{Acc} \leftarrow \mathbf{Acc} + \mathbf{A} \cdot \mathbf{B}$ 
13:   end for
14:   if  $S_k = 1$  then
15:     Store  $\mathbf{Acc}$  to  $\mathbf{y}$  at non- $\perp$  worklist coordinates
16:   else
17:     Atomic-add  $\mathbf{Acc}$  to  $\mathbf{y}$  at non- $\perp$  worklist coordinates
18:   end if
19: end for

```

Algorithm 2 summarizes the persistent compute model. The structure mirrors a standard tiled implicit-GEMM convolution (outer loop over output $\times$ channel tiles, inner loop over spatial and channel reduction), with two differences: the outer loop is grid-stride rather than one-to-one with blocks, and the M -dimension indices come from the worklist rather than a contiguous range.

Properties. The persistent compute path introduces no host synchronization (the grid is occupancy-sized, the worklist length is read on-device), retains a dense-compatible inner datapath along the channel and kernel dimensions, and inherits tile-local spatial structure from construction’s tile-ordered emission. §5 reports the achieved throughput relative to the dense path. However, the configuration is fixed at build time, while the worklist length it must serve varies widely across frames and layers. The next subsection makes the decomposition data-driven.

3.3.2 Adaptive Workload Decomposition. The tile configuration (B_M, B_N, B_K, S_k) determines how work is decomposed; B_N and B_K participate in calibration but do not couple to η , because η is defined over position slots alone: channel-axis quantization (C_{out} not divisible by B_N) wastes work inside a slot and so enters t , lowering T rather than η . The following analysis focuses on B_M and S_k . B_M simultaneously governs two axes through Eq. 5. A larger B_M amortizes

shared-memory loads (higher per-slot efficiency, benefiting T) but widens the packing gap when $n_{\mathcal{W}}$ is comparable to B_M (lower η_{pack}). At low scheduled density, a smaller B_M raises both η_{pack} and the number of independent work items; at high density, $n_{\mathcal{W}} \gg B_M$ makes η_{pack} near 1 and a larger B_M maximizes per-slot throughput. Split-K adds parallelism along the reduction dimension at the cost of atomic accumulation and a zeroing pass over scheduled coordinates; it helps when output tiles alone underfill the device, but its cost is a net loss at high density. A single fixed configuration cannot serve both regimes: what a configuration must maximize is effective throughput ηT , not either factor alone. Within this choice η_{cov} is fixed by construction and reuse, so for worklist candidates the objective reduces to $\eta_{\text{pack}} T$, while the dense candidate enters with $\eta = \alpha$.

Selecting the best configuration at runtime requires knowing each module’s scheduled density, yet reading it from the device per layer would reintroduce the synchronization the persistent design eliminates. WISEConv resolves this tension through offline calibration combined with a sync-free online feedback loop.

Offline calibration. Levels index the *scheduled* density $n_{\mathcal{W}}/|\mathcal{G}|$ rather than the exact active ratio $\alpha = \|M_{\text{out}}\|_0/|\mathcal{G}|$: the scheduled length is what determines compute cost and η_{pack} . Under reuse the two diverge ($\eta_{\text{cov}} < 1$ means $n_{\mathcal{W}} > \|M_{\text{out}}\|_0$), and calibration targets the scheduled density to match compute’s actual workload. The range is discretized into N_ℓ levels (we use $N_\ell = 10$, spanning scheduled density from 100% to ~10%); densities below the lowest calibrated level use that level’s configuration. For each convolution shape in the model and each level, a one-time calibration pass generates a synthetic worklist of the corresponding length that preserves the local grouping property produced by construction, sweeps all candidates, and records the fastest measured one. A *candidate* is an executable choice for the operator: a worklist tile configuration, or the dense path described below. The result is a per-shape, per-level lookup table $\mathcal{L}$; this table is computed once at deployment, persisted to disk, and reduces runtime candidate selection to a single table lookup.

Dense candidate. For operators whose shape cuDNN supports (standard and grouped convolutions with supported padding modes), the candidate set includes the dense cuDNN path, which issues the full grid ($\eta = \alpha$). Let T_{wl} and T_{dense} denote the per-slot throughput of the worklist and dense paths respectively. Their effective throughputs are $\eta_{\text{cov}} \eta_{\text{pack}} T_{\text{wl}}$ and αT_{dense} , so the dense candidate is faster when

$$T_{\text{dense}}/T_{\text{wl}} > \eta_{\text{cov}} \eta_{\text{pack}} / \alpha :$$

its per-slot throughput advantage, which comes from loading contiguous input tiles instead of gathering by coordinate, must exceed the worklist path’s η advantage. Substituting the definitions gives $\eta_{\text{cov}} \eta_{\text{pack}} / \alpha = |\mathcal{G}|/|C|$: the worklist

path's η advantage is the ratio of the full grid to the slots it actually issues. The advantage shrinks as the scheduled density approaches one, and on a small grid B_M padding can carry $|C|$ past $|G|$ ($|G| = 400$, $n_W = 390$, $B_M = 128$ issues 512 slots), at which point the dense path wins on both factors at once. Under reuse the scheduled density is governed by the leading construction rather than by the operator's own α , so a reused operator can face a near-unit or sub-unit ratio while its own active ratio is low. Independently, T_{wl} itself degrades at small n_W : too few independent work items leave the device below capacity (§3.3.1). The η ratio follows from n_W , B_M , and $|G|$ by arithmetic, but the throughput ratio T_{dense}/T_{wl} does not follow from the shape, so WISEConv locates the crossover by measurement rather than by a cost model.

Application mapping. The feedback loop requires two conditions. The policy must supply an input mask whose active positions propagate along receptive fields (Eq. 1), so that each module's scheduled density follows from that mask rather than from an independent per-layer prediction. Successive inputs must also be temporally correlated, so that an observation lagged by one or more frames still approximates the current density. Under both conditions, a second offline pass profiles the deployed model on representative sequences, recording per-module scheduled density as a function of the input-activity bucket. Because this pass runs the model with its reuse decisions already fixed, the densities it records are the scheduled densities compute will actually see, matching what the calibration table indexes. The result is the per-module mapping $\{M_i\}$ that translates a single input-activity observation into a per-module density level: different layers amplify input activity differently depending on their receptive-field size and mask-propagation depth. $\{M_i\}$ relies on the empirical regularity that per-module scheduled density correlates with global input activity across frames of a given workload.

Sync-free activity feedback. WISEConv delivers the current input activity to the host without stalling the compute stream via an asynchronous readback on a dedicated copy stream. After each frame's input mask is computed, the runtime enqueues an asynchronous device-to-host transfer of the input activity count (active positions in the network's input grid, $H_{in}W_{in}$), gated by an event so the copy waits until the count is ready. A ring buffer $\mathcal{R}$ of N_s slots ($N_s = 4$) absorbs variation in transfer latency; if all slots are in flight, the current observation is dropped without blocking.

At the start of the next frame, completed slots are polled without synchronization. The most recent completed observation maps to an input-activity bucket, whose offline mapping supplies a density level for each module. If no new observation has completed, execution proceeds with the current configuration rather than waiting.

Algorithm 3 Nonblocking Temporal Configuration Selection

Require: Ring buffer $\mathcal{R}$ (N_s slots), per-module mapping $\{M_i\}$, calibration table $\mathcal{L}$, modules $\{m_i\}$, input grid size $H_{in}W_{in}$

Ensure: Per-module candidate updated for current frame

```

1: // Poll: consume latest completed input-activity
   observation
2:  $\hat{a} \leftarrow \text{ASYNCPOLL}(\mathcal{R})$  {non-blocking}
3: if  $\hat{a}$  is available then
4:    $b \leftarrow \text{BUCKET}(\hat{a} / H_{in}W_{in})$ 
5:   for each convolution module  $m_i$  do
6:      $\ell_i \leftarrow M_i[b]$  {per-module level from application
       mapping}
7:      $q_i \leftarrow \mathcal{L}[m_i.\text{shape}, \ell_i]$  {worklist config or dense
       candidate}
8:      $m_i.\text{SETCANDIDATE}(q_i)$ 
9:   end for
10: end if
11: // Execute frame
12:  $a_{in} \leftarrow$  run input mask generation {device-resident input
   active count}
13: if  $\mathcal{R}$  has a free slot then
14:    $\text{ASYNCEQUEUE}(\mathcal{R}, a_{in})$  {async copy, waits on
     mask-generation event}
15: end if
16: Run remaining construction and compute for all modules

```

Algorithm 3 summarizes the feedback loop for the event and temporal workloads (FireFlowNet, YOLOv8), which satisfy both conditions. Learned spatial gating [34] fails the first regardless of the input: each gated block predicts its own mask from its own features, and the network input stays fully dense, so the count the loop would read is constant across frames. WISEConv therefore selects fixed per-layer levels for gated models from the calibrated gate distributions, rather than tracking each module's own density, which would replace one readback per frame with one per module. The applied configuration is lagged by one or more frames relative to the current input. This lag does not affect correctness: every candidate in the calibrated set satisfies the coverage contract (§2.1) independently of the activity estimate. A worklist candidate reads its exact device-resident length as the work bound; the dense candidate computes the full grid while the exact mask still governs which outputs update downstream state. Feedback latency therefore affects decomposition quality, not execution completeness. §5 quantifies the resulting throughput benefit.

4 Implementation

We implement WISEConv as a CUDA C++ extension for PyTorch. The current backend executes FP32 tensors in channels-last layout and provides the mask-aware operators required

by the evaluated networks: convolution, transposed convolution, pooling, pointwise activation, addition, concatenation, and upsampling. A model-conversion pass replaces compatible PyTorch modules while preserving the graph topology, convolution parameters, and learned weights. The upstream application continues to generate masks.

The construction kernel assigns contiguous output positions to each warp. A thread tests one receptive field and terminates once it finds an active input. A warp ballot identifies active lanes, population counts give their local ranks, and one atomic operation reserves a contiguous worklist segment for the warp. Active lanes then write their coordinates in lane order. This realizes the tile-local ordering in Section 3.2 without a device-wide prefix scan or sorting pass. An identity 1×1 convolution has the same input and output coordinate space; when its input already carries compatible mask and worklist metadata, the operator reuses that worklist and skips construction.

The compute kernel uses a persistent, tiled implicit-GEMM organization. It keeps partial sums in registers and stages input and weight tiles through shared memory. Supported architectures use asynchronous global-to-shared copies, with a synchronous path for earlier GPUs. Split- K exposes additional parallel work when the active-coordinate dimension alone cannot occupy the device. Transposed convolution follows the same output-driven organization, but partitions output coordinates by stride phase so that each phase visits only its valid kernel offsets.

Efficient scheduling depends jointly on the GPU, layer shape, and activity. We therefore autotune the sparse tile shape and split- K factor and cache the choice by device, convolution parameters, activity level, and execution policy. For layers whose state-update semantics permit dense evaluation, the candidate set can also include a cuDNN path. A sparse-only policy isolates the worklist-driven kernel, and the cache key distinguishes the two policies and the requested determinism mode. Autotuning and one-time weight preparation complete before the timed sequence described in Section 5.1.

For fixed input values, weights, and mask, WISEConv evaluates Eq. (2) at every active output coordinate. Floating-point reordering may produce small numerical differences from cuDNN, so we verify active outputs with numerical tolerances rather than requiring bitwise identity. This check separates backend correctness from the task-quality effect of the fixed upstream mask policy.

5 Evaluation

5.1 Experimental Setup

We evaluate WISEConv across four perception models, three mask sources, four GPU platforms, and three competing execution paradigms. The evaluation asks four questions: (1) does WISEConv reduce full-model latency relative to the

fastest available path, (2) how effectively does it convert required-work reduction into latency reduction, (3) what overhead does worklist construction add, and (4) does replacing the masked-convolution backend change task output or accuracy?

5.1.1 Platforms. We evaluate four representative GPU platforms: two embedded SoCs (Jetson Xavier NX and Jetson AGX Orin), a laptop-class mobile GPU (RTX 4070 Laptop), and a desktop GPU (RTX 3080). These devices cover the hardware classes commonly used for onboard deployment and robotics development. The two Jetson systems anchor our target deployment setting, while the discrete GPUs test whether WISEConv generalizes beyond embedded SoCs.

5.1.2 Models, Tasks, and Mask Sources. Our selected workloads span three robotic perception tasks:

- **Event-based optical flow.** We evaluate FireFlowNet [25], a lightweight event-based optical-flow model that delivers high inference rates while retaining competitive accuracy, on MVSEC [42]. MVSEC contains event-camera sequences and corresponding ground-truth flow from indoor quadrotor flight and outdoor driving. Following Ev-Conv [7], consecutive 50-ms windows advance by 1 ms; events entering or leaving the window form the sparse increment from which we derive layer masks. We calibrate the sparsification policy on indoor_flying1 and evaluate on the other three sequences.
- **Video object detection.** We evaluate YOLOv8n and YOLOv8m [18] on MOT16 [23], a collection of crowded pedestrian videos with annotated person boxes. Both models produce bounding boxes and confidence scores, while their different scales expose how model size affects sparse-execution efficiency. DeltaCNN [27] supplies the temporal mask mechanism by propagating thresholded activation deltas between successive frames.
- **Human pose estimation.** We use the DynConv pose model [34] on MPII [1], which depicts people performing diverse activities and annotates their 2D body joints. The model predicts joint heatmaps; its learned spatial gates provide input-dependent masks at gated blocks.

Table 1 summarizes their models and evaluation scales. We use official weights for all four models [18, 25, 34]. For each workload, we fix these weights, inputs, and upstream mask policy; the sparse paths receive the same resulting layer masks and differ only in their masked-convolution backend.

5.1.3 Execution Baselines. We compare WISEConv against three FP32 execution paths established by prior work.

- **Dense** uses the native PyTorch/cuDNN backend [5] with sparse execution disabled.

Table 1. Evaluation workloads.

	FireFlowNet [25]	YOLOv8n/m [18]	DynConv Pose [34]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [7]	DeltaCNN [27]	DynConv [34]
Dataset	MVSEC [42]	MOT16 [23]	MPII [1]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,575	2,958

- **Tile skipping** follows the tile-sparse execution established by SBNet, Ev-Conv, and DeltaCNN [7, 27, 29].
- **Gather-scatter** uses the original DynConv CUDA backend for human pose [34] and an implementation equivalent to MMCV MaskedConv2d for FireFlowNet and YOLO [4].

5.1.4 Metrics and Methodology. Our primary metric is full-model GPU latency. The timed region includes mask propagation, worklist construction, all network operators, and output update, but excludes host-to-device input transfer, calibration, autotuning, and the one-time dense state initialization required by incremental models. We use CUDA events on the model’s compute stream and process complete held-out sequences rather than isolated synthetic frames. Each sequence is repeated three times; we retain per-frame samples and report both aggregate latency and dispersion.

To explain latency, we report the required-work ratio ρ_{work} , useful-compute ratio η , convolution throughput, and the construction fraction of operator latency. Throughput here isolates the compute kernel; construction is reported separately as a fraction of operator latency, so readers can reconstruct the inclusive T of §2.2. We aggregate η as total active FLOPs over total scheduled FLOPs across all convolution layers, so each layer contributes in proportion to the work it actually schedules. Because layers differ in cost, an unweighted average of layer activities misstates the work a mask removes, so we weight each layer by its dense MAC count,

$$\rho_{\text{work}} = \frac{\sum_l \alpha^l W_{\text{dense}}^l}{\sum_l W_{\text{dense}}^l}, \quad \alpha^l = \frac{\|M^l\|_0}{H_{\text{out}}^l W_{\text{out}}^l}, \quad (6)$$

where W_{dense}^l is the dense MAC count of layer l . Task quality is measured with Average Endpoint Error (AEE) for optical flow, detection AP against the fixed evaluation references for YOLO, and PCKh for human pose. Accuracy comparisons use the same masks and operating point as latency measurements; they evaluate the upstream policy, while operator correctness is checked separately against dense convolution at every active output.

5.2 Full-Model Latency

5.3 Efficiency Analysis

5.4 Construction Overhead

5.5 Task Accuracy

References

- [1] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.
- [2] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.
- [3] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBInfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.
- [4] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).
- [5] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).
- [6] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [7] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. Ev-Conv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.
- [8] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739
- [9] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117
- [10] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712
- [11] RuiBo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024– 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafirir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378
- [12] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.
- [13] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.
- [14] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings*

- of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728
- [15] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [16] Amirhossein Habibi, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.
- [17] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838
- [18] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>
- [19] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).
- [20] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.
- [21] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348
- [22] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.
- [23] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. *CoRR abs/1603.00831* (2016).
- [24] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR abs/2104.08378* (2021).
- [25] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.
- [26] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.
- [27] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.
- [28] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426
- [29] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.
- [30] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. <https://github.com/traveller59/spconv>.
- [31] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.
- [32] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [33] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.
- [34] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.
- [35] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.
- [36] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.
- [37] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.
- [38] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.
- [39] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.
- [40] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [41] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.
- [42] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR abs/1801.10202* (2018).